

AlgorithmPart-03

Insertion sort

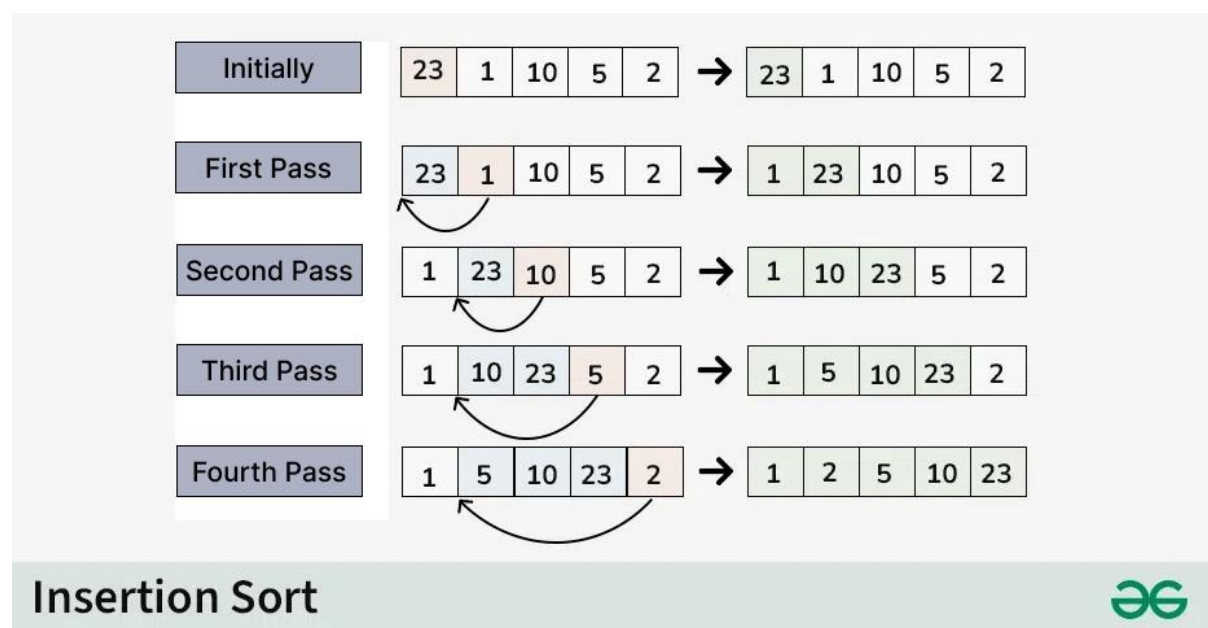
Insertion sort is a simple sorting algorithm that works by iteratively inserting each element of an unsorted list into its correct position in a sorted portion of the list. It is a stable sorting algorithm, meaning that elements with equal values maintain their relative order in the sorted output.

Steps to Perform Insertion Sort:

- **Start with the Second Element:** Assume the first element is already sorted. Begin with the second element.
- **Compare and Swap:** Compare the second element with the first:
 - If the second element is smaller, swap them.
- **Move to the Next Element:** Take the third element and compare it with the ones before it (the second and first):
 - Swap as needed to place it in the correct order.
- **Repeat the Process:** Continue this for each element in the array:
 - Compare the current element with the sorted elements before it.
 - Swap until the current element is in its correct position.
- **Finish When All Elements Are Sorted:** Keep repeating until you've gone through the entire array.

Working of Insertion Sort Algorithm:

Consider an array having elements : {23, 1, 10, 5, 2}



Initial:

- Current element is 23
- The first element in the array is assumed to be sorted.
- The sorted part until 0th index is : [23]

First Pass:

- Compare 1 with 23 (current element with the sorted part).
- Since 1 is smaller, insert 1 before 23 .
- The sorted part until 1st index is: [1, 23]

Second Pass:

- Compare 10 with 1 and 23 (current element with the sorted part).
- Since 10 is greater than 1 and smaller than 23 , insert 10 between 1 and 23 .
- The sorted part until 2nd index is: [1, 10, 23]

Third Pass:

- Compare 5 with 1 , 10 , and 23 (current element with the sorted part).
- Since 5 is greater than 1 and smaller than 10 , insert 5 between 1 and 10
- The sorted part until 3rd index is : [1, 5, 10, 23]

Fourth Pass:

- Compare 2 with 1, 5, 10 , and 23 (current element with the sorted part).
- Since 2 is greater than 1 and smaller than 5 insert 2 between 1 and 5 .
- The sorted part until 4th index is: [1, 2, 5, 10, 23]

Final Array:

- The sorted array is: [1, 2, 5, 10, 23]

Pseudocode

```
for j = 2 to A.length  
    key = A[j]  
    i = j - 1  
    while i > 0 and A[i] > key  
        A[i + 1] = A[i]  
        i = i - 1  
    A[i + 1] = key
```

Complexity Analysis of Insertion Sort :

Time Complexity of Insertion Sort

- **Best case: $O(n)$** , If the list is already sorted, where n is the number of elements in the list.
- **Average case: $O(n^2)$** , If the list is randomly ordered
- **Worst case: $O(n^2)$** , If the list is in reverse order

Q1. What are the Boundary Cases of the Insertion Sort algorithm?

Insertion sort takes the maximum time to sort if elements are sorted in reverse order. And it takes minimum time (Order of n) when elements are already sorted.

Q2. What is the Algorithmic Paradigm of the Insertion Sort algorithm?

The Insertion Sort algorithm follows an incremental approach.

Q3. Is Insertion Sort an in-place sorting algorithm?

Yes, insertion sort is an in-place sorting algorithm.

Q4. Is Insertion Sort a stable algorithm?

Yes, insertion sort is a stable sorting algorithm.

Q5. When is the Insertion Sort algorithm used?

Insertion sort is used when number of elements is small. It can also be useful when the input array is almost sorted, and only a few elements are misplaced in a complete big array.

Bubble Sort

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity is quite high.

Bubble Sort Algorithm

- traverse from left and compare adjacent elements and the higher one is placed at right side.
- In this way, the largest element is moved to the rightmost end at first.
- This process is then continued to find the second largest and place it and so on until the data is sorted.

How does Bubble Sort Work?

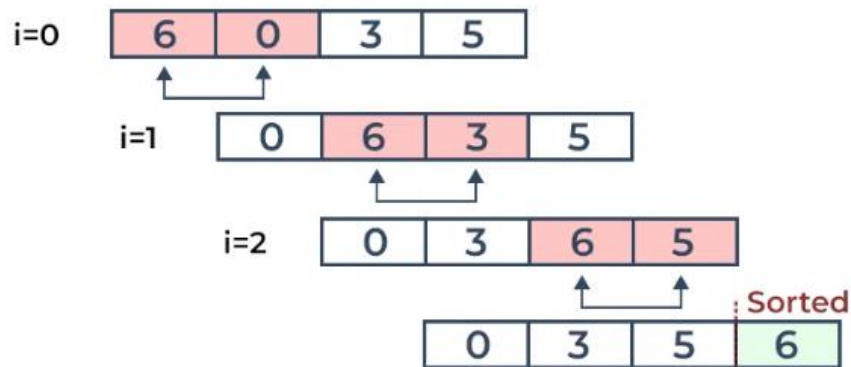
Let us understand the working of bubble sort with the help of the following illustration: **Input:** arr[] = {6, 0, 3, 5}

First Pass:

The largest element is placed in its correct position, i.e., the end of the array.

STEP
01

Placing the 1st largest element at Correct position



Bubble sort

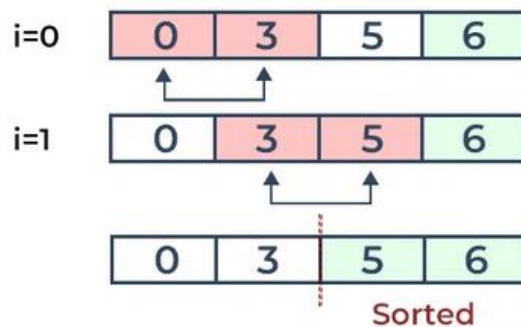


Second Pass:

Place the second largest element at correct position

STEP
02

Placing 2nd largest element at Correct position



Bubble sort

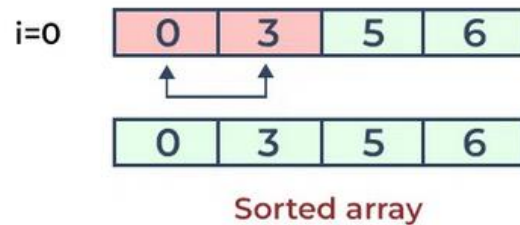


Third Pass:

Place the remaining two elements at their correct positions.

STEP
03

Placing 3rd largest element at Correct position



Bubble sort



- Total no. of passes: $n-1$
- Total no. of comparisons: $n*(n-1)/2$

Complexity Analysis of Bubble Sort:

TimeComplexity: $O(N^2)$

Auxiliary Space: $O(1)$

Pseudocode

Initialize n = Length of Array

BubbleSort(Array, n)

for $i = 0$ to $n-2$

for $j = 0$ to $n-2$

if $\text{Array}[j] > \text{Array}[j+1]$

swap($\text{Array}[j]$, $\text{Array}[j+1]$)

Some FAQs related to Bubble Sort:

What is the Boundary Case for Bubble sort?

Bubble sort takes minimum time (Order of n) when elements are already sorted. Hence it is best to check if the array is already sorted or not beforehand, to avoid $O(N^2)$ time complexity.

Does sorting happen in place in Bubble sort?

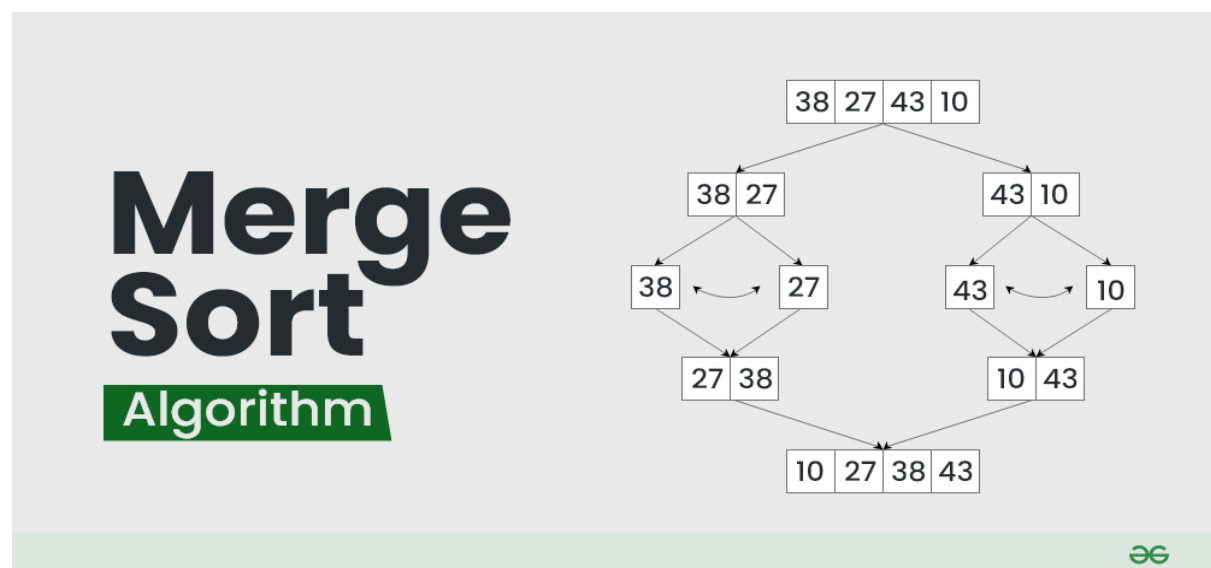
Yes, Bubble sort performs the swapping of adjacent pairs without the use of any major data structure. Hence Bubble sort algorithm is an in-place algorithm.

Is the Bubble sort algorithm stable?

Yes, the bubble sort algorithm is stable.

Merge sort

Merge sort is a sorting algorithm that follows the **divide-and-conquer** approach. It works by recursively dividing the input array into smaller subarrays and sorting those subarrays then merging them back together to obtain the sorted array.



How does Merge Sort work?

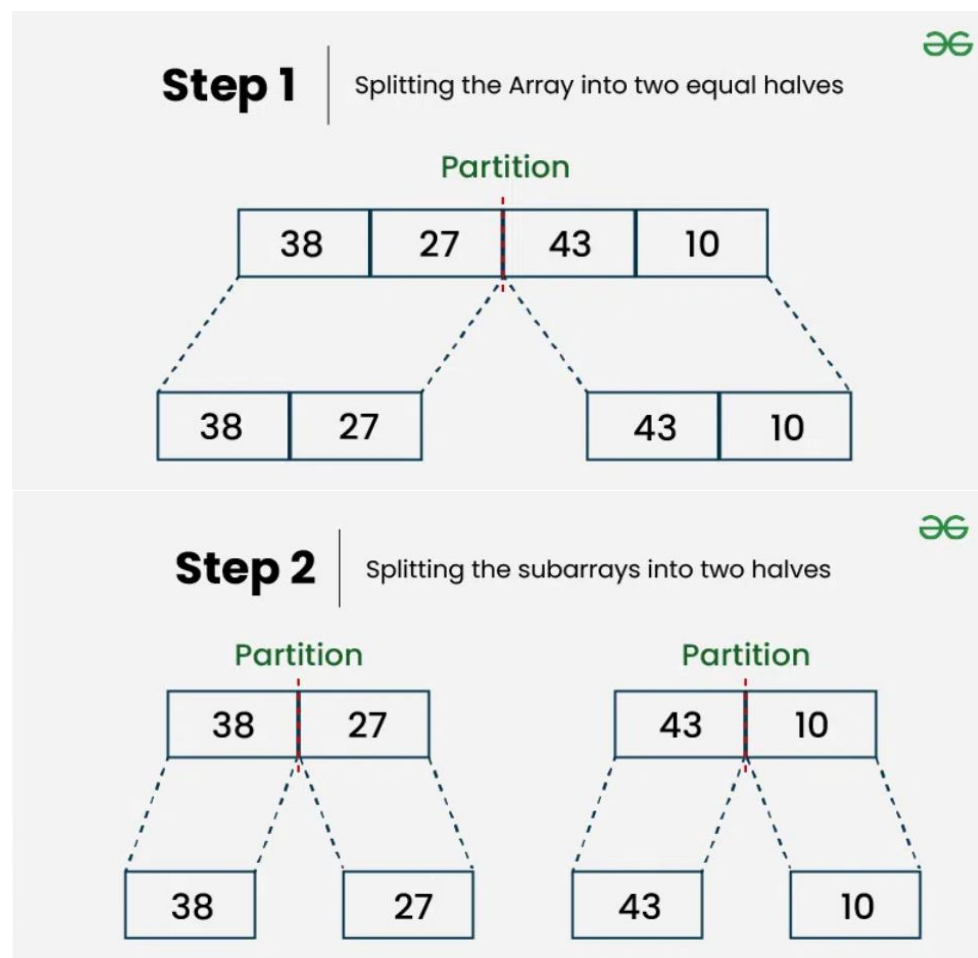
Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the **divide-and-conquer** approach to sort a given array of elements.

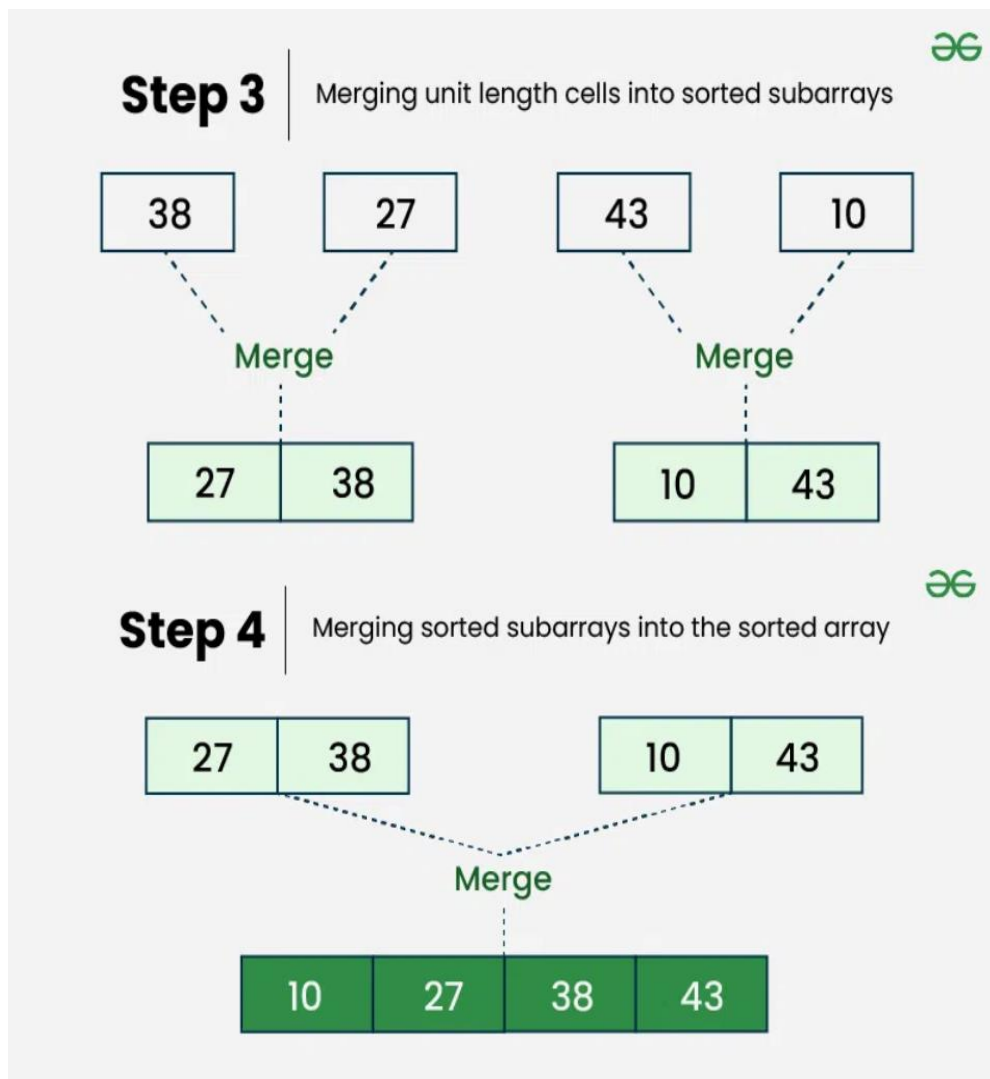
Here's a step-by-step explanation of how merge sort works:

1. **Divide:** Divide the list or array recursively into two halves until it can no more be divided.
2. **Conquer:** Each subarray is sorted individually using the merge sort algorithm.
3. **Merge:** The sorted subarrays are merged back together in sorted order. The process continues until all elements from both subarrays have been merged.

Illustration of Merge Sort:

Let's sort the array or list [38, 27, 43, 10] using Merge Sort





Let's look at the working of above example:

Divide:

- [38, 27, 43, 10] is divided into [38, 27] and [43, 10] .
- [38, 27] is divided into [38] and [27] .
- [43, 10] is divided into [43] and [10] .

Conquer:

- [38] is already sorted.
- [27] is already sorted.

- [43] is already sorted.
- [10] is already sorted.

Merge:

- Merge [38] and [27] to get [27, 38] .
- Merge [43] and [10] to get [10,43] .
- Merge [27, 38] and [10,43] to get the final sorted list [10, 27, 38, 43]

Therefore, the sorted list is [10, 27, 38, 43] .

Pseudocode

MergeSort(array, left, right)

if left < right

mid = (left + right) / 2

MergeSort(array, left, mid)

MergeSort(array, mid + 1, right)

Merge(array, left, mid, right)

Merge(array, left, mid, right)

leftArray = array[left to mid]

rightArray = array[mid+1 to right]

i, j, k = 0, 0, left

while i < size(leftArray) and j < size(rightArray)

if leftArray[i] <= rightArray[j]

array[k++] = leftArray[i++]

else

array[k++] = rightArray[j++]

copy remaining elements to array

Complexity Analysis of Merge Sort:

- **Time Complexity:**
 - Best Case: $O(n \log n)$, When the array is already sorted or nearly sorted.
 - Average Case: $O(n \log n)$, When the array is randomly ordered.
 - Worst Case: $O(n \log n)$, When the array is sorted in reverse order.
- **Auxiliary Space:** $O(n)$, Additional space is required for the temporary array used during merging.

Quick Sort

QuickSort is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array.

Time Complexity:

- Best Case : $\Omega(N \log(N))$
The best-case scenario for quicksort occur when the pivot chosen at the each step divides the array into roughly equal halves.
In this case, the algorithm will make balanced partitions, leading to efficient Sorting.
- Average Case: $\theta(N \log(N))$
Quicksort's average-case performance is usually very good in practice, making it one of the fastest sorting Algorithm.
- Worst Case: $O(N^2)$
The worst-case Scenario for Quicksort occur when the pivot at each step consistently results in highly unbalanced partitions
- Auxiliary Space: $O(1)$, if we don't consider the recursive stack space. If we consider the recursive stack space then, in the worst case quicksort could make $O(N)$.

Pseudocode

Partition(array, low, high)

 pivot = array[low]

 i = low

 j = high

 while i < j

 while array[i] <= pivot and i < high

 i = i + 1

 while array[j] > pivot

 j = j - 1

 if i < j

 swap(array[i], array[j])

 swap(array[low], array[j])

 return j

QuickSort(array, low, high)

 if low < high

 j = Partition(array, low, high)

 QuickSort(array, low, j - 1)

 QuickSort(array, j + 1, high)

Types of Asymptotic Notations in Complexity Analysis of Algorithms

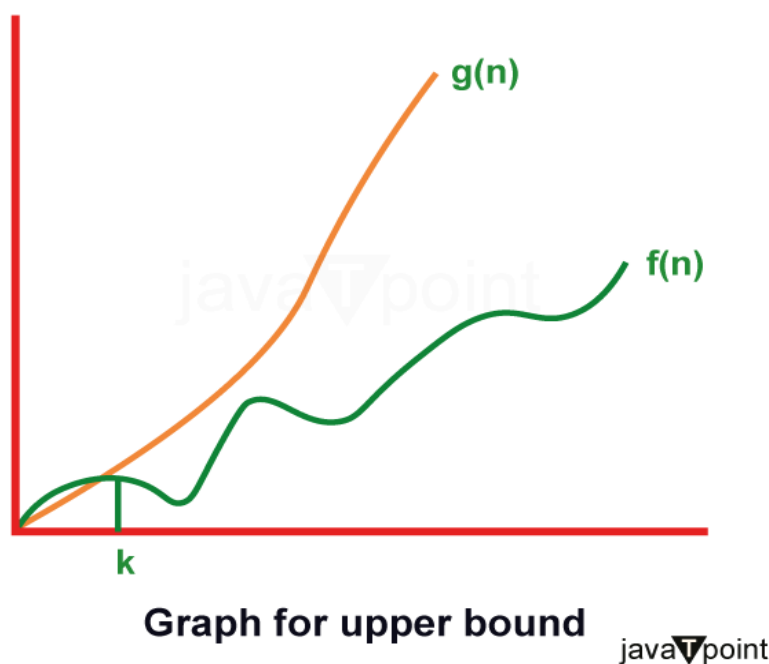
There are mainly three asymptotic notations:

1. Big-O Notation (O-notation)
2. Omega Notation (Ω -notation)
3. Theta Notation (Θ -notation)

1. Big-oh notation: Big-oh is the formal method of expressing the upper bound of an algorithm's running time. It is the measure of the longest amount of time. The function $f(n) = O(g(n))$ [read as "f of n is big-oh of g of n"] if and only if exist positive constant c and such that

$$f(n) \leq k \cdot g(n) \text{ for } n > n_0 \text{ in all case}$$

Hence, function $g(n)$ is an upper bound for function $f(n)$, as $g(n)$ grows faster than $f(n)$



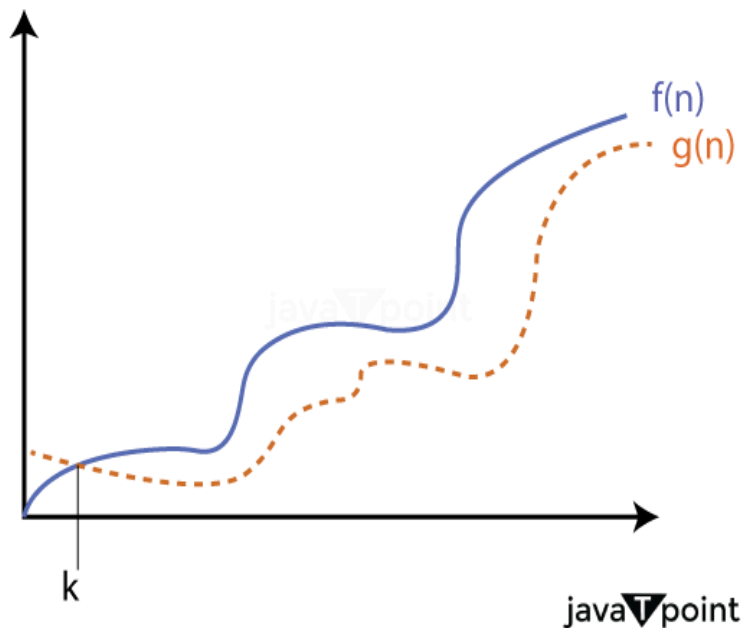
For Example:

1. $3n+2=O(n)$ as $3n+2 \leq 4n$ for all $n \geq 2$
2. $3n+3=O(n)$ as $3n+3 \leq 4n$ for all $n \geq 3$

Hence, the complexity of $f(n)$ can be represented as $O(g(n))$

2. Omega () Notation: The function $f(n) = \Omega(g(n))$ [read as "f of n is omega of g of n"] if and only if there exists positive constant c and n_0 such that

$f(n) \geq k * g(n)$ for all $n, n \geq n_0$



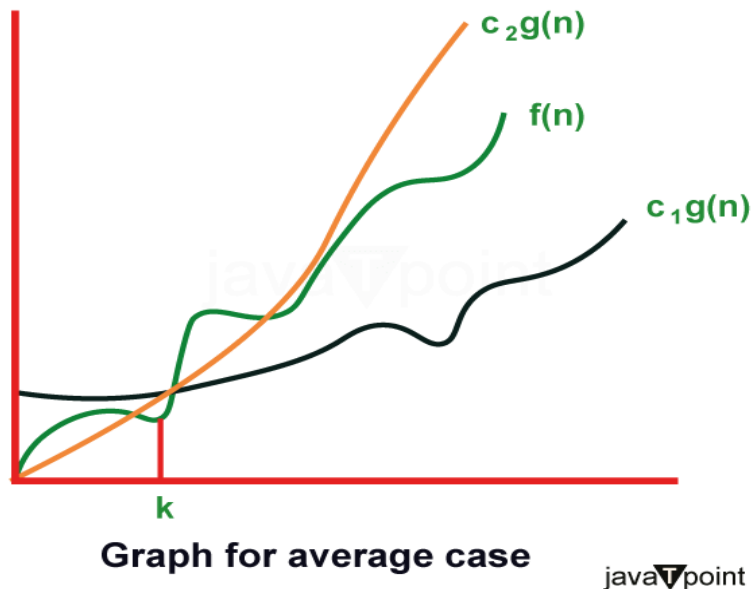
For Example:

$$\begin{aligned} f(n) &= 8n^2 + 2n - 3 \geq 8n^2 - 3 \\ &= 7n^2 + (n^2 - 3) \geq 7n^2 (g(n)) \end{aligned}$$

Thus, $k_1 = 7$

Hence, the complexity of $f(n)$ can be represented as $\Omega(g(n))$

3. Theta (θ): The function $f(n) = \theta(g(n))$ [read as "f is the theta of g of n"] if and only if there exists positive constant k_1 , k_2 and k_0 such that
 $k_1 * g(n) \leq f(n) \leq k_2 g(n)$ for all n , $n \geq n_0$



For Example:

$3n+2 = \theta(n)$ as $3n+2 \geq 3n$ and $3n+2 \leq 4n$, for n
 $k_1=3, k_2=4$, and $n_0=2$

Hence, the complexity of $f(n)$ can be represented as $\theta(g(n))$.

The Theta Notation is more precise than both the big-oh and Omega notation.

The function $f(n) = \theta(g(n))$ if $g(n)$ is both an upper and lower bound.

Rod Cutting

Given a rod of length n inches and an array of prices that includes prices of all pieces of size smaller than n . Determine the maximum value obtainable by cutting up the rod and selling the pieces. We are mainly given a price array such that the size of the price array is same as rod length and **price[i] represents price of length $i+1$**

Input : price[] = { 1 , 5, 8, 9, 10, 17 , 17 , 20}

Output : 22

Explanation : Rod length is 8 and the values of different pieces are given as the following,

length | 1 2 3 4 5 6 7 8

price | 1 5 8 9 10 17 17 20

The maximum obtainable value is 22 (by cutting in two pieces of lengths 2 and 6)

Input : price[] = { 3 , 5, 8, 9, 10, 17}

Output : 18

Explanation : Rod length is 7 And if the prices are as follows.

length | 1 2 3 4 5 6

price | 3 4 6 7 9 10

The maximum obtainable value is 18 (by making 6 pieces each of length 1)

Input : price[] = { 3 }

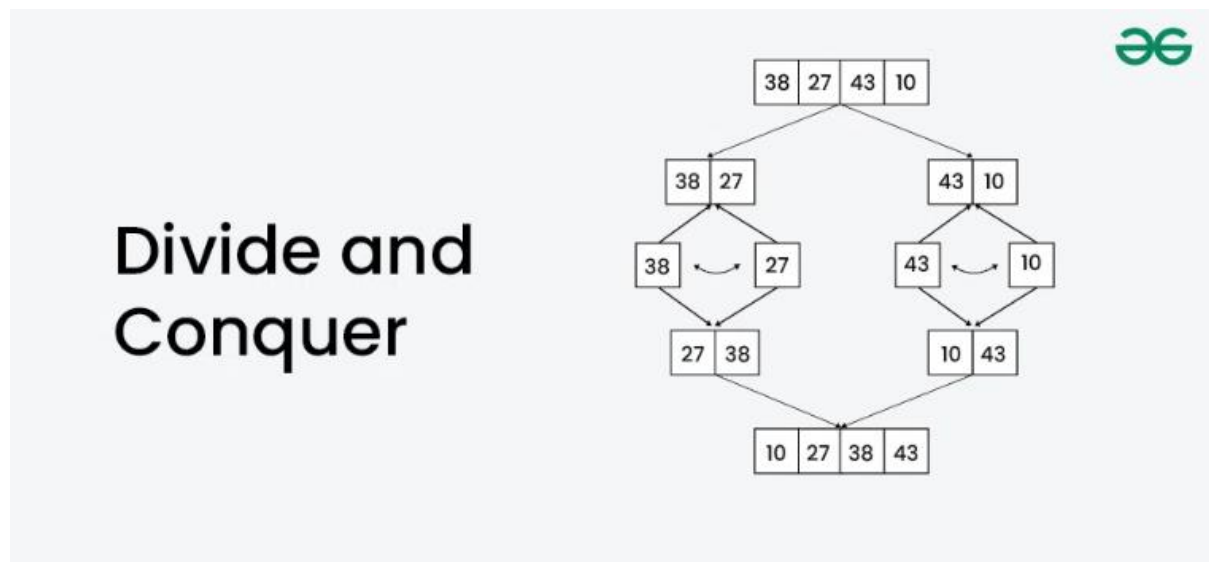
Output : 3

Explanation: There is only 1 way to pick a piece of length 1

Divide and Conquer Algorithm

Divide and Conquer algorithm is a problem-solving strategy that involves breaking down a complex problem into smaller, more manageable parts, solving each part individually, and then combining the solutions to solve the original problem. It is a widely used algorithmic technique in computer science and mathematics.

Example: In the Merge Sort algorithm, the “Divide and Conquer” strategy is used to sort a list of elements. Below image illustrate the dividing and merging states to sort the array using Merge Sort.



Properties of Divide and Conquer:

- Divides the problem into smaller subproblems.
- Each subproblem is solved independently.
- Solutions of subproblems are combined to obtain the solution to the original problem.

Examples of Divide and Conquer Algorithms:

There are various algorithms that follow the divide-and-conquer algorithm to solve a problem efficiently. Some common examples are given below:

- **Merge Sort:** It is a sorting algorithm that uses the divide and conquers approach to sort an array of elements.
- **Binary Search:** It is a search algorithm that uses the divide and conquers approach to find an element in a sorted array.
- **Quick Sort:** It is also a sorting algorithm that uses the divide and conquers approach to sort an array of elements.

1.What is Divide and Conquer Algorithm?

Divide and Conquer is a problem-solving technique that involves breaking a larger problem into subproblems, solving the subproblems independently and combining the solutions of those subproblems to get the solution of the larger problem.

2.Stages of Divide and Conquer Algorithm:

Divide and Conquer Algorithm can be divided into three stages: **Divide**, **Conquer** and **Merge**.

1. Divide:

- Break down the original problem into smaller subproblems.
- Each subproblem should represent a part of the overall problem.
- The goal is to divide the problem until no further division is possible.

2. Conquer:

- Solve each of the smaller subproblems individually.
- If a subproblem is small enough (often referred to as the “base case”), we solve it directly without further recursion.
- The goal is to find solutions for these subproblems independently.

3. Merge:

- Combine the sub-problems to get the final solution of the whole problem.

- Once the smaller subproblems are solved, we recursively combine their solutions to get the solution of larger problem.
- The goal is to formulate a solution for the original problem by merging the results from the subproblems.

Applications of Divide and Conquer Algorithm

1. **Merge Sort:** This sorting method divides an array into smaller parts, sorts each part, and then merges them back together to create a sorted array.
2. **Median Finding:** You can find the median of a group of numbers by repeatedly splitting the group into smaller parts, making the search faster.
3. **Min and Max Finding:** This technique helps find the smallest and largest numbers in an array at the same time by dividing the array into halves and comparing results.
4. **Matrix Multiplication:** Strassen's algorithm uses divide and conquer to multiply large matrices more efficiently by breaking them into smaller matrices and combining the results.
5. **Closest Pair Problem:** This problem involves finding the two nearest points in a set. A divide and conquer approach splits the points into groups and then combines the results to find the closest pair efficiently.

Advantages of Divide and Conquer:

- It provides an efficient solution to complicated problems.
- It can solve more difficult problems than other approaches.
- It reduces the complexity of problems by breaking them down into smaller ones.
- It can be used for a wide range of problems across various domains.

Disadvantages of Divide and Conquer:

- It can be more difficult to implement than other algorithms.
- It is not much useful for small problems.

- It requires additional time and space complexity for splitting up and merging data.

Heap sort

Heap sort is a comparison-based sorting technique based on Binary Heap Data Structure. It can be seen as an optimization over selection sort where we first find the max (or min) element and swap it with the last (or first). We repeat the same process for the remaining elements. In Heap Sort, we use Binary Heap so that we can quickly find and move the max element (In $O(\log n)$ instead of $O(n)$) and hence achieve the $O(n \log n)$ time complexity.

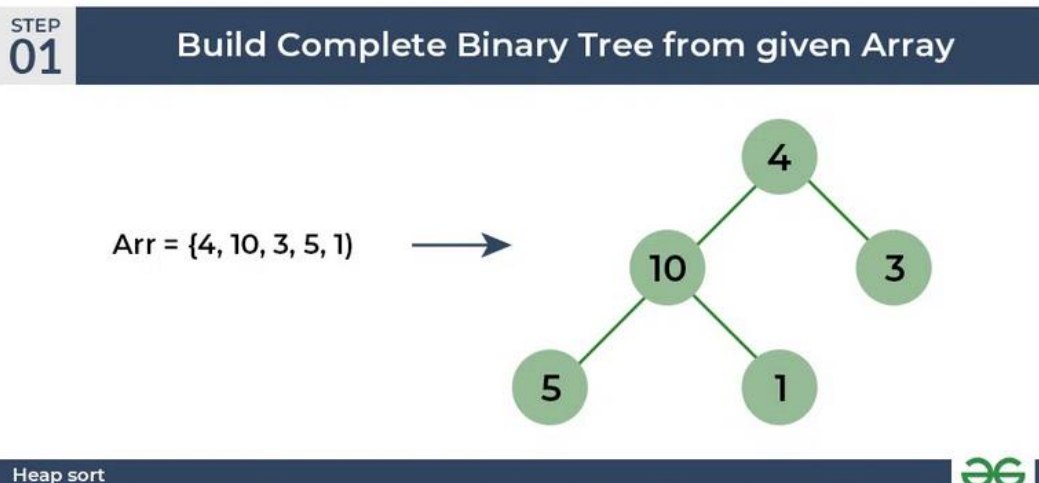
Heap Sort Algorithm

1. Build a max heap from the given input array.
2. Repeat the following steps until the heap contains only one element:
 - a. Swap the root element of the heap (which is the largest element) with the last element of the heap.
 - b. Remove the last element of the heap (which is now in the correct position).
 - c. Heapify the remaining elements of the heap.
3. The sorted array is obtained by reversing the order of the elements in the input array.

Detailed Working of Heap Sort

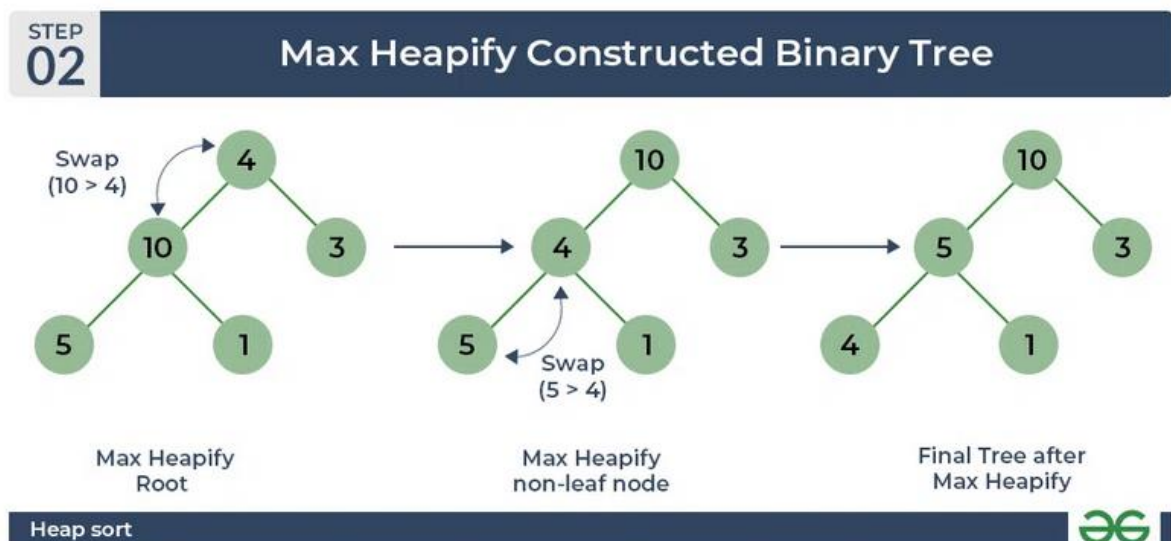
Consider the array: `arr[] = {4, 10, 3, 5, 1}`.

Consider the given array as Complete Binary Tree (For every index i , the left child is at index $2*i + 1$ and right at $2*i + 2$).



Transform the array into max heap:

- To transform a heap into a max-heap, the parent node should always be greater than or equal to the child nodes
 - Here, in this example, as the parent node **4** is smaller than the child node **10**, thus, swap them to build a max-heap.
- Now, **4** as a parent is smaller than the child **5**, thus swap both of these again and the resulted heap and array should be like this:

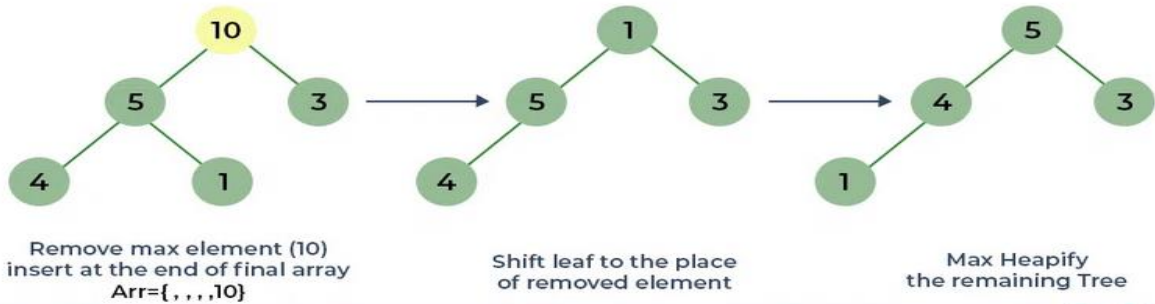


Perform heap sort: Remove the maximum element in each step (i.e., move it to the end position and remove that) and then consider the remaining elements and transform it into a max heap.

- Delete the root element (**10**) from the max heap. In order to delete this node, try to swap it with the last node, i.e. (**1**). After removing the root element, again heapify it to convert it into max heap.
 - Resulted heap and array should look like this:

STEP
03

Remove Maximum from Root and Max Heapify



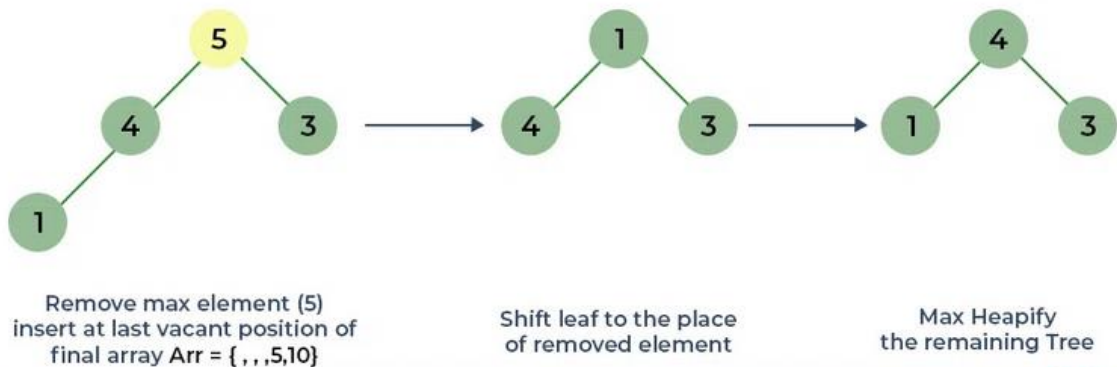
Heap sort



Repeat the above steps and it will look like the following:

STEP
04

Remove Next Maximum from Root and Max Heapify



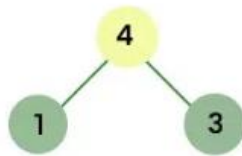
Heap sort



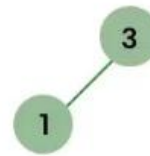
Now remove the root (i.e. 4) again and perform heapify.

STEP
05

Remove Next Maximum from Root and Max Heapify



Remove max element (3)
insert at last vacant position
of final array $Arr = \{, 3, 4, 5, 10\}$



Shift leaf to the place
of removed element.
(No heapify needed)

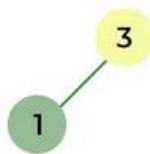
Heap sort



Now remove the root (i.e. 3) again and perform heapify.

STEP
06

Remove Next Maximum from Root and Max Heapify



Remove max element (3)
insert at last vacant position
of final array $Arr = \{, 3, 4, 5, 10\}$



Shift leaf to the place
of removed element.
(No heapify needed)

Heap sort



Now when the root is removed once again it is sorted. and the sorted array will be like $arr[] = \{1, 3, 4, 5, 10\}$.

STEP
07

Remove Last Element and Return Sorted Array

1

→ Arr =

1	3	4	5	10
---	---	---	---	----

Remove max element (1)
Arr = {1, 3, 4, 5, 10}

Final sorted array

Heap sort



Complexity Analysis of Heap Sort

Time Complexity: $O(n \log n)$

Auxiliary Space: $O(\log n)$, due to the recursive call stack. However, auxiliary space can be $O(1)$ for iterative implementation.

1. What are the two phases of Heap Sort?

The heap sort algorithm consists of two phases. In the first phase, the array is converted into a max heap. And in the second phase, the highest element is removed (i.e., the one at the tree root) and the remaining elements are used to create a new max heap.

2. Is Heap Sort an example of the “Divide and Conquer” algorithm?

Heap sort is **NOT** at all a Divide and Conquer algorithm. It uses a heap data structure to efficiently sort its element and not a “divide and conquer approach” to sort the elements.

3. Which sorting algorithm is better – Heap sort or Merge Sort?

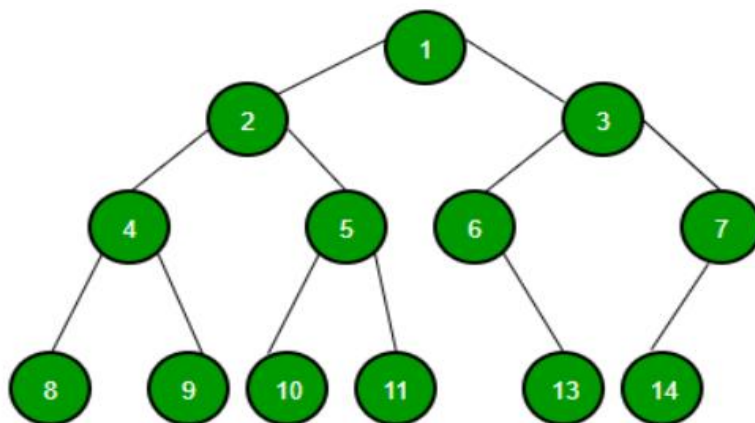
The answer lies in the comparison of their time complexity and space requirements. The Merge sort is slightly faster than the Heap sort. But on the other hand merge sort takes extra memory. Depending on the requirement, one should choose which one to use.

4. Why is Heap sort better than Selection sort?

Heap sort is similar to selection sort, but with a better way to get the maximum element. It takes advantage of the heap data structure to get the maximum element in constant time

Binary Tree

A **Binary Tree Data Structure** is a hierarchical data structure in which each node has at most two children, referred to as the left child and the right child. It is commonly used in computer science for efficient storage and retrieval of data, with various operations such as insertion, deletion, and traversal.

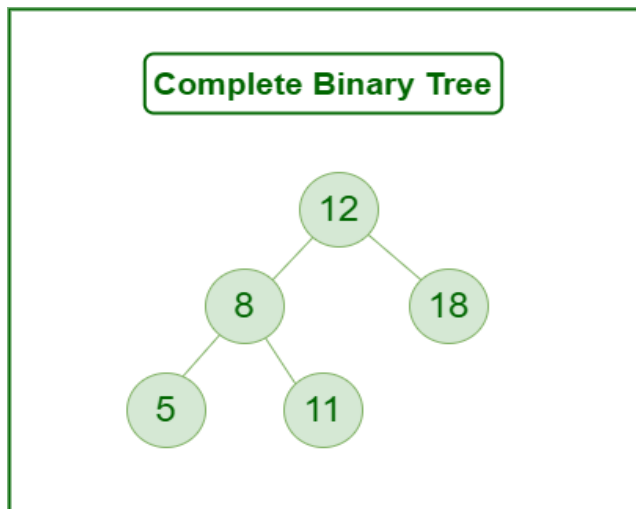


Types of Binary Tree On the basis of the completion of levels:

1. Complete Binary Tree
2. Perfect Binary Tree
3. Balanced Binary Tree

1. Complete Binary Tree

A Binary Tree is a Complete Binary Tree if all the levels are completely filled except possibly the last level and the last level has all keys as left as possible.

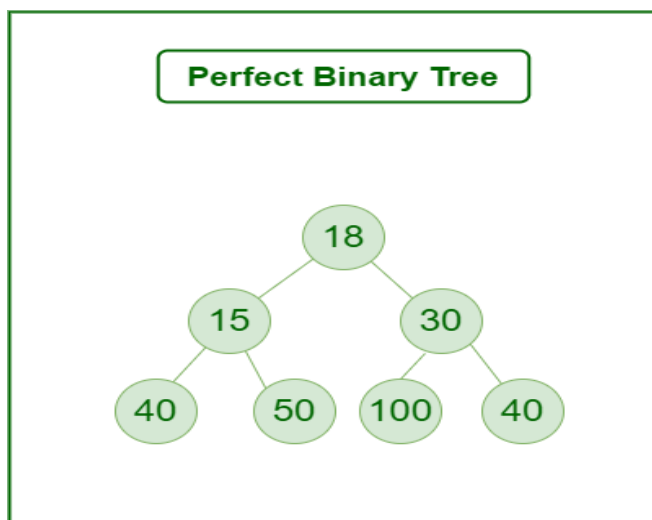


Complete Binary Tree

2. Perfect Binary Tree

A Binary tree is a Perfect Binary Tree in which all the internal nodes have two children and all leaf nodes are at the same level.

The following are examples of Perfect Binary Trees.



Perfect Binary Tree

In a Perfect Binary Tree, the number of leaf nodes is the number of internal nodes plus 1

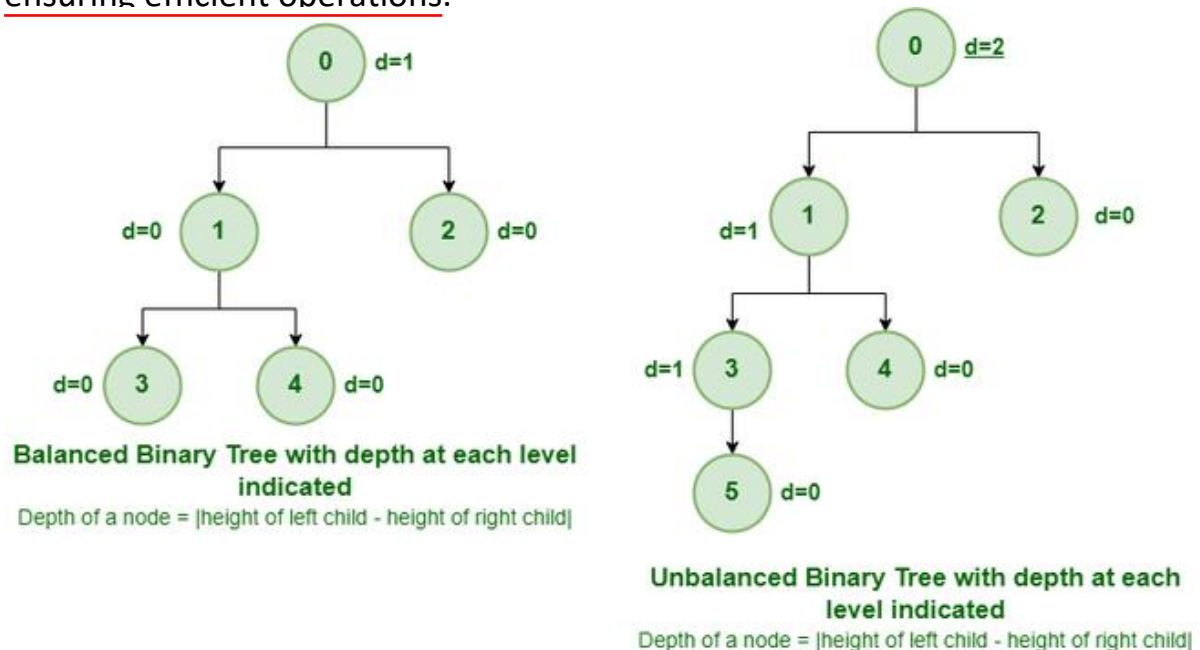
$L = I + 1$ Where L = Number of leaf nodes, I = Number of internal nodes.

A Perfect Binary Tree of height h (where the height of the binary tree is the number of edges in the longest path from the root node to any leaf node in the tree, height of root node is 0) has $2^{h+1} - 1$ node.

An example of a Perfect binary tree is ancestors in the family. Keep a person at root, parents as children, parents of parents as their children.

3. Balanced Binary Tree

A balanced binary tree is a binary tree where the height difference between the left and right subtrees of any node is no more than one, ensuring efficient operations.



Example of Balanced and Unbalanced Binary Tree

It is a type of binary tree in which the difference between the height of the left and the right subtree for each node is either 0 or 1. In the figure above, the root node having a value 0 is unbalanced with a depth of 2 units.

When to Use Dynamic Programming (DP)

Dynamic Programming is useful when:

1. **Overlapping Subproblems:** The problem can be broken down into smaller, overlapping subproblems that can be solved independently. Solutions to these subproblems can be reused.
2. **Optimal Substructure:** The optimal solution to the problem can be constructed from the optimal solutions of its subproblems.

3. Examples:

- **Fibonacci Sequence:** Instead of calculating each Fibonacci number from scratch, use previously computed values.
- **Knapsack Problem (0/1 Knapsack):** Use DP to find the maximum value that can be carried without exceeding the weight limit.
- **Longest Common Subsequence:** Finding the longest subsequence common to two sequences.

When to Use Greedy Algorithms

Greedy Algorithms are suitable when:

1. **Local Optimal Choice:** The problem can be solved by making a series of choices, each of which is locally optimal. Making the best choice at each step leads to a global optimal solution.
2. **No Overlapping Subproblems:** The problem does not have overlapping subproblems and does not require revisiting past decisions.

3. Examples:

- **Activity Selection Problem:** Choose the maximum number of activities that don't overlap by selecting the one that finishes the earliest.
- **Huffman Coding:** Build an optimal prefix code for data compression by combining the least frequent items first.
- **Minimum Spanning Tree (Prim's and Kruskal's Algorithms):** Construct a tree that connects all vertices with the least total edge weight.

Algorithms

An **algorithm** is a step-by-step set of instructions to solve a problem or perform a task. Algorithms can be simple or complex, depending on the problem they are solving.

Basic Notations of Algorithms

When we talk about algorithms, we often use some basic notations to describe their behavior:

1. **Input:** The data you provide to the algorithm.
2. **Output:** The result produced by the algorithm after processing the input.
3. **Steps:** The specific operations or actions the algorithm takes to transform the input into the output.
4. **Control Structures:** These include loops (like for and while) and conditionals (like if statements) that control the flow of the algorithm.

Criteria of Algorithms

When evaluating algorithms, you can consider several criteria:

1. **Correctness:** Does the algorithm give the right answer for all inputs?
2. **Efficiency:** How fast does it run? How much memory does it use?
3. **Clarity:** Is the algorithm easy to understand and follow? A clear algorithm is easier to maintain and debug.
4. **Generality:** Can the algorithm work for a wide range of inputs, or is it too specific?
5. **Optimality:** Is the algorithm the best possible solution in terms of time and space?

Fundamental Analysis of Complexity

In computer science, complexity analysis is crucial for understanding the efficiency of algorithms. It helps determine how the time or space requirements of an algorithm grow as the input size increases.

Types of Complexity

1. **Time Complexity:** The amount of time an algorithm takes to complete as a function of the length of the input.
2. **Space Complexity:** The amount of memory space required by the algorithm in relation to the input size.

Asymptotic Notation

Asymptotic notation is used to describe the behavior of functions as the input size grows. The most common types are:

1. **Big O Notation (O):** Represents the upper bound of the time complexity. It describes the worst-case scenario.
 - Example: $O(n)$ indicates that the time taken grows linearly with the input size.
2. **Omega Notation (Ω):** Represents the lower bound of the time complexity. It describes the best-case scenario.
 - Example: $\Omega(n)$ suggests that the algorithm takes at least linear time.
3. **Theta Notation (Θ):** Represents a tight bound on the time complexity. It implies that the time grows at the same rate in both the worst and best cases.
 - Example: $\Theta(n)$ means both the upper and lower bounds are linear.

Efficiency

Efficiency in algorithms refers to how well an algorithm utilizes resources, particularly time and space. A more efficient algorithm will solve problems quicker and with less memory usage.

Case Analysis

1. **Worst Case:** The maximum time taken by an algorithm for any input of size n . It ensures that the algorithm will not exceed this time for any input.
 - Example: In a linear search, the worst-case time complexity is $O(n)$

2. **Best Case:** The minimum time taken by an algorithm for any input of size n . It shows the scenario where the algorithm performs the least amount of work.
 - Example: In a linear search, the best-case time complexity is $O(1)$ if the desired element is the first one checked.
3. **Average Case:** The expected time taken by an algorithm over all possible inputs of size n . This is often more difficult to calculate but provides a realistic view of performance.
 - Example: For a linear search, the average-case time complexity is $O(n)$, assuming the element is equally likely to be at any position.

Trees

A tree is a hierarchical data structure made up of nodes connected by edges, with a single root node and no cycles. Each node can have zero or more child nodes, forming a parent-child relationship.

Types of Trees

1. Binary Tree:

- Each node has at most two children, referred to as the left and right child.
- **Properties:**
 - The height of a binary tree is the number of edges from the root to the deepest leaf.
 - Traversal methods include in-order, pre-order, and post-order.

2. Binary Search Tree (BST):

- A special type of binary tree where the left child of a node contains values less than the parent node, and the right child contains values greater.

- **Operations:**
 - Insertion, deletion, and lookup operations can be performed efficiently, typically in $O(\log n)$ time if the tree is balanced.
- **Use Cases:** Implementing dynamic sets and lookup tables.

3. **Balanced Trees:**

- Trees that maintain a balanced height to ensure efficient operations.
- **Examples:**
 - **AVL Trees:** A self-balancing binary search tree where the difference in heights of left and right subtrees is at most one.

Use Cases

- **Hierarchical Data Representation:** Organizing data in a structure that reflects parent-child relationships, such as file systems or organizational charts.
- **Searching and Sorting Operations:** Efficiently finding and sorting data elements using methods like tree traversal.

Graphs

A **graph** is a collection of nodes (also known as vertices) connected by edges. Graphs can represent various relationships and structures in real-world scenarios.

Types of Graphs

1. **Directed Graph:**
 - Each edge has a direction, indicating a one-way relationship from one vertex to another.
 - **Use Cases:** Representing web pages (links) or social networks (following relationships).

2. **Undirected Graph:**

- Edges do not have a direction; the relationship is mutual.
- **Use Cases:** Modeling friendships in social networks or connections in a network.

3. **Weighted Graph:**

- Edges have weights or costs associated with them, representing the strength or distance of the connection.
- **Use Cases:** Navigating transportation systems where weights could represent distances, travel times, or costs.

Use Cases

- **Social Networks:** Representing users as vertices and relationships (friendships, follows) as edges.
- **Transportation Systems:** Modeling routes and distances between locations, allowing for efficient pathfinding.
- **Network Routing:** Finding the shortest path in communication networks, using algorithms like Dijkstra's or Bellman-Ford

Linear Search Algorithm

Linear search is a method for searching for an element in a collection of elements. In linear search, each element of the collection is visited one by one in a sequential fashion to find the desired element. Linear search is also known as sequential search.

Algorithm for Linear Search Algorithm:

The algorithm for linear search can be broken down into the following steps:

- **Start:** Begin at the first element of the collection of elements.
- **Compare:** Compare the current element with the desired element.
- **Found:** If the current element is equal to the desired element, return true or index to the current element.
- **Move:** Otherwise, move to the next element in the collection.

- **Repeat:** Repeat steps 2-4 until we have reached the end of collection.
- **Not found:** If the end of the collection is reached without finding the desired element, return that the desired element is not in the array.

How does linear search algorithm work?

Linear search algorithm iterates through each element in the list or array, comparing it with the target element until a match is found or the end of the list is reached. If the end of the list is reached, then it means that the target element is not present in the array.

For example: Consider the array `arr[] = {10, 50, 30, 70, 80, 20, 90, 40}` and `key = 30`

Step 1: Start from the first element (index 0) and compare **key** with each element (`arr[i]`).

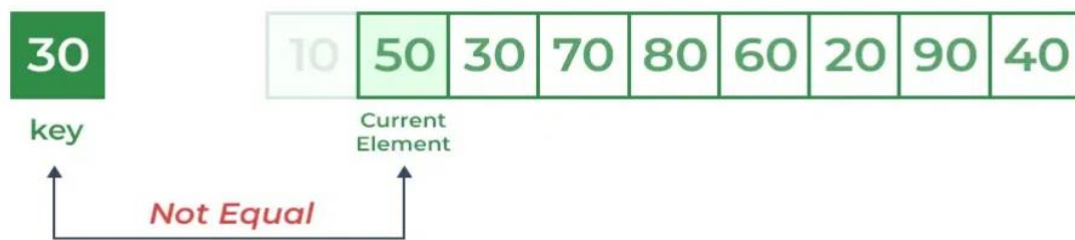
- Comparing key with first element `arr[0]`. Since not equal, the iterator moves to the next element as a potential match.



Linear Search Algorithm



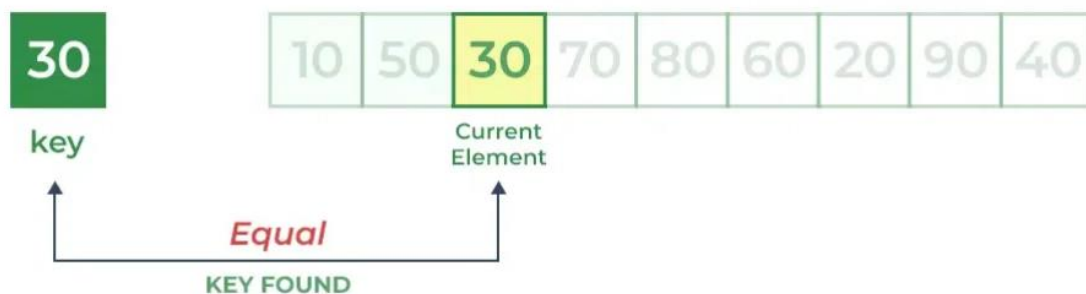
Comparing key with next element `arr[1]`. Since not equal, the iterator moves to the next element as a potential match.



Linear Search Algorithm



Step 2: Now when comparing arr[2] with key, the value matches. So the Linear Search Algorithm will yield a successful message and return the index of the element when key is found (here 2).



Linear Search Algorithm



When to use Linear Search Algorithm?

- When we are dealing with a small dataset.
- When you are searching for a dataset stored in contiguous memory.

Time Complexity:

- **Best Case:** In the best case, the key might be present at the first index. So the best case complexity is $O(1)$
- **Worst Case:** In the worst case, the key might be present at the last index i.e., opposite to the end from which the search has started in the list. So the worst-case complexity is $O(N)$ where N is the size of the list.
- **Average Case:** $O(N)$

Binary Search Tree

A Binary Search Tree is a data structure used in computer science for organizing and storing data in a sorted manner. Each node in a Binary Search Tree has at most two children, a left child and a right child, with the left child containing values less than the parent node and the right child containing values greater than the parent node. This hierarchical structure allows for efficient searching, insertion, and deletion operations on the data stored in the tree.

